

Free RAG System

Complete Line-by-Line Code Explainer

Covers every import, every function, and every key line in `create_database.py` and `query_data.py` — written so you can explain each piece confidently, e.g. in a technical interview.

Legend:

Built-in (comes with Python itself, no install needed)

Third-party (installed via pip / requirements.txt)

0. The Big Picture

This project has two separate scripts because RAG has two separate jobs:

- **create_database.py** — runs ONCE (or whenever your source PDFs change). Reads the PDF, splits it into chunks, converts each chunk into a vector (embedding), and saves everything into a local Chroma database on disk. No AI text-generation happens here — just math (turning text into numbers) and file storage.
- **query_data.py** — runs EVERY TIME you ask a question. Turns your question into a vector the same way, searches the saved database for the closest matching chunks, then sends those chunks + your question to an LLM (via Groq) to generate a real, natural-language answer.

1. create_database.py — Imports

1.1

```
from langchain_community.document_loaders import PyPDFLoader
```

THIRD-PARTY (pip install langchain-community)

Where it comes from: Part of the LangChain project. LangChain itself is not built into Python — it's a framework you install with pip, specifically to make it easy to build LLM/RAG apps like this one.

Why we use it here: PyPDFLoader is a ready-made class that opens a PDF file and extracts the raw text from every page, wrapping each page as a LangChain 'Document' object (text + metadata like the source filename and page number). Without this, you'd have to manually use a lower-level PDF library and write that page-splitting logic yourself.

1.2

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

THIRD-PARTY (pip install langchain-text-splitters)

Where it comes from: Also part of the LangChain ecosystem, split into its own package in newer LangChain versions (in older versions this lived inside the main 'langchain' package).

Why we use it here: This class breaks long text into small overlapping chunks. 'Recursive' means it tries to split on natural boundaries first (paragraphs, then sentences, then words) rather than cutting mid-sentence wherever it hits a character limit. Chunking matters because you can't feed an entire 250-page book into the vector search or the LLM at once — it needs to be broken into small, independently-searchable pieces.

1.3

```
from langchain_huggingface import HuggingFaceEmbeddings
```

THIRD-PARTY (pip install langchain-huggingface)

Where it comes from: A LangChain integration package that wraps HuggingFace's open-source sentence-embedding models so they plug directly into LangChain's interfaces.

Why we use it here: This is the FREE replacement for OpenAI's paid embeddings. It loads a small open-source model (we use 'all-MiniLM-L6-v2') that runs entirely on your own machine and converts any piece of text into a vector — a list of numbers representing that text's meaning.

1.4

```
from langchain_community.vectorstores import Chroma
```

THIRD-PARTY (pip install langchain-community, chromadb)

Where it comes from: A LangChain wrapper around the separate 'chromadb' library, so Chroma can be used with LangChain's standard vector-store interface.

Why we use it here: Chroma is the actual vector database: it stores every chunk's vector and lets you search 'which stored vectors are closest to this new vector?' very quickly. This is what makes retrieval fast even across a full book.

1.5

```
import os
```

BUILT-IN (comes with Python, no install needed)

Where it comes from: Part of Python's standard library — installed automatically with Python itself.

Why we use it here: Used here to list files in a folder (os.listdir) and to build safe file paths (os.path.join) that work on both Windows and Mac/Linux.

1.6

```
import shutil
```

BUILT-IN (comes with Python, no install needed)

Where it comes from: Also part of Python's standard library.

Why we use it here: Used for one thing here: shutil.rmtree(), which deletes an entire folder (the old Chroma database) so we can rebuild it fresh each time the script runs, instead of mixing old and new data.

1. create_database.py — Functions, Line by Line

main()

```
def main():  
    generate_data_store()
```

The entry point. 'def' defines a function named main that takes no arguments. It does nothing except call generate_data_store() — this keeps main() simple and makes the file readable top-to-bottom: 'main runs the whole pipeline.'

```
if __name__ == "__main__":  
    main()
```

A standard Python pattern. __name__ is a special built-in variable Python sets automatically. It equals "__main__" only when you run this file directly (python create_database.py) — NOT when this file is imported by another script. This lets you safely import functions from this file elsewhere without main() auto-running.

generate_data_store()

```
def generate_data_store():  
    documents = load_documents()  
    chunks = split_text(documents)  
    save_to_chroma(chunks)
```

The 3-step pipeline in one place: load the PDF(s) into memory as 'documents', split those documents into small 'chunks', then save those chunks (as embeddings) into Chroma. Each step's output feeds directly into the next step's input — this is the core data flow of the whole file.

load_documents()

```
all_documents = []
```

Creates an empty Python list. [] is Python's syntax for a list — an ordered, changeable collection. We'll fill this with every page from every PDF.

```
for filename in os.listdir(DATA_PATH):
```

os.listdir() returns the names of every file in the DATA_PATH folder ("data/books"). This loop runs once per file found.

```
if filename.lower().endswith(".pdf"):
```

.lower() converts the filename to lowercase so "MyFile.PDF" still matches. .endswith(".pdf") checks the file extension. This filters out any non-PDF files (like a stray .txt or .DS_Store) that might be in the folder.

```
path = os.path.join(DATA_PATH, filename)
loader = PyPDFLoader(path)
all_documents.extend(loader.load())
```

os.path.join builds the full file path correctly for whatever operating system you're on. PyPDFLoader(path) creates a loader object pointed at that file. .load() actually opens the PDF and returns a list of Document objects (one per page). .extend() adds all of those page-documents onto our running all_documents list (as opposed to .append(), which would nest them as a single sub-list — we want one flat list of pages).

```
return all_documents
```

Sends the finished list of page-documents back to whoever called this function (generate_data_store).

split_text(documents)

```
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=100,
    length_function=len,
    add_start_index=True,
)
```

Creates the splitter object with settings: chunk_size=500 means each chunk targets ~500 characters. chunk_overlap=100 means each chunk repeats the last 100 characters of the previous chunk, so a sentence that gets cut at a chunk boundary still has surrounding context in both neighboring chunks. length_function=len tells it to measure length in raw characters (as opposed to, say, tokens). add_start_index=True stores each chunk's starting character position from the original document, useful for citing back to an exact location later.

```
chunks = text_splitter.split_documents(documents)
```

Runs the actual splitting. Takes the list of page-documents in, returns a (usually much longer) list of small chunk-documents out — each original page typically becomes several chunks.

```
if chunks:
    sample = chunks[min(10, len(chunks) - 1)]
    print(sample.page_content)
    print(sample.metadata)
```

A safety + debug step. 'if chunks:' only runs this block if the list isn't empty (an empty list is treated as False in Python). chunks[min(10, len(chunks)-1)] safely grabs the 11th chunk (index 10) for a preview, or the last chunk if there are fewer than 11 — the min() prevents an 'index out of range' crash on a very small document. Printing it just lets you visually sanity-check what a chunk of text (and its metadata, like source file and page number) actually looks like.

```
return chunks
```

Returns the full list of chunk-documents to `generate_data_store()`.

save_to_chroma(chunks)

```
if os.path.exists(CHROMA_PATH):  
    shutil.rmtree(CHROMA_PATH)
```

Checks whether a Chroma database folder already exists from a previous run, and if so, deletes it completely. This guarantees a clean rebuild every time — otherwise old and new chunks could pile up together.

```
embedding_function = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
```

Loads the free local embedding model. The first time this runs, it downloads the model (~90MB) from HuggingFace's servers and caches it locally; every run after that loads it from your own disk, no internet needed for this step.

```
db = Chroma.from_documents(  
    chunks, embedding_function, persist_directory=CHROMA_PATH  
)
```

This single line does the heavy lifting: for every chunk, it runs the `embedding_function` to turn that chunk's text into a vector, then stores the (text + vector + metadata) as a record inside Chroma. `persist_directory` tells Chroma to save all of this to disk (the 'chroma/' folder) rather than just keeping it in memory, so `query_data.py` can load it back later.

```
print(f"Saved {len(chunks)} chunks to '{CHROMA_PATH}'.")
```

An f-string (the `f` before the quotes) lets you embed variables directly inside a string using {curly braces}. Just a confirmation message so you know the run succeeded and how many chunks were saved.

2. query_data.py — Imports

2.1

```
import argparse
```

BUILT-IN (comes with Python, no install needed)

Where it comes from: Python standard library.

Why we use it here: Lets the script accept a command-line argument — the question you type after the filename, e.g. `python query_data.py "who is Harry?"`. Without this, you'd have to hard-code the question inside the file itself.

2.2

```
import os
```

BUILT-IN

Where it comes from: Python standard library.

Why we use it here: Used to read the `GROQ_API_KEY` value out of your computer's environment variables (`os.environ`), after `dotenv` loads it from the `.env` file.

2.3

```
from dotenv import load_dotenv
```

THIRD-PARTY (pip install python-dotenv)

Where it comes from: A small, popular utility package (not part of Python itself).

Why we use it here: Reads your local `.env` file and loads its `KEY=VALUE` lines into the program's environment variables at runtime. This is how your Groq API key gets from the `.env` file into `os.environ`, without ever hard-coding the key into the Python file itself (which would be a security risk if you shared the code, e.g. on GitHub).

2.4

```
from langchain_community.vectorstores import Chroma
```

THIRD-PARTY (pip install langchain-community, chromadb)

Where it comes from: Same package as in `create_database.py`.

Why we use it here: Used here to RE-OPEN the existing Chroma database that `create_database.py` already built and saved to disk, so we can search it.

2.5

```
from langchain_huggingface import HuggingFaceEmbeddings
```

THIRD-PARTY (pip install langchain-huggingface)

Where it comes from: Same package as in `create_database.py`.

Why we use it here: Must use the exact same embedding model here as when the database was built. Vectors from two different embedding models aren't comparable — it would be like measuring some chunks in miles and others in kilometers and then asking which numbers are closest.

2.6

```
from langchain_groq import ChatGroq
```

THIRD-PARTY (pip install langchain-groq)

Where it comes from: LangChain's official integration package for Groq's API.

Why we use it here: This is the class that actually talks to Groq's servers over the internet to run the LLM (currently `openai/gpt-oss-20b`) and get back a generated answer. This is the **ONLY** paid-capable step in the whole pipeline — but we're using it on Groq's free tier.

2.7

```
from langchain_core.prompts import ChatPromptTemplate
```

THIRD-PARTY (pip install langchain-core, a dependency of langchain)

Where it comes from: The foundational LangChain package that other LangChain packages build on.

Why we use it here: A template engine for building prompts. Instead of manually gluing strings together with + signs, you write a template once with {placeholders} (like {context} and {question}) and then call `.format()` to fill them in — cleaner and less error-prone.

2.8

```
from colorama import init, Fore, Style
```

THIRD-PARTY (pip install colorama)

Where it comes from: A small cross-platform package for colored terminal text.

Why we use it here: Windows terminals historically didn't support color codes the same way Mac/Linux terminals do. `colorama.init()` patches this so the same color codes work everywhere. `Fore.GREEN` and `Style.BRIGHT` are then used to print the final answer in a color that stands out from the plain debug logs.

2. query_data.py — Line by Line

Setup (top-level code)

```
init(autoreset=True)
```

Activates colorama. `autoreset=True` means the color automatically resets back to normal after each `print()`, so you don't have to manually reset the color every time or risk your whole terminal turning green forever.

```
load_dotenv()
```

Actually reads the `.env` file in your project folder and loads its variables (like `GROQ_API_KEY`) so `os.environ` can see them. This must run before you try to read `os.environ["GROQ_API_KEY"]` later, or it won't exist yet.

```
CHROMA_PATH = "chroma"
```

A constant (by convention, ALL_CAPS names signal 'this shouldn't change') storing the folder name where the Chroma database lives. Must match the same `CHROMA_PATH` used in `create_database.py`, or this script will look in the wrong place and find nothing.

```
PROMPT_TEMPLATE = """
Answer the question based only on the following context...
{context}
---
...{question}
"""
```

The strict-mode instruction given to the LLM. Triple-quotes ("""") let a string span multiple lines. `{context}` and `{question}` are placeholders that get filled in later with the retrieved chunks and your actual question. The phrase 'based only on the following context' is what forces strict mode — the model is told not to rely on its own outside knowledge.

main()

```
parser = argparse.ArgumentParser()
parser.add_argument("query_text", type=str, help="The question to ask.")
args = parser.parse_args()
query_text = args.query_text
```

Sets up command-line argument parsing. `add_argument` defines one required argument named `query_text`, expected to be a string (`type=str`). `parse_args()` reads whatever you actually typed after 'python query_data.py' on the command line. `args.query_text` then holds that text — e.g. "who is Harry?" — as a normal Python string.

```
embedding_function = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
db = Chroma(persist_directory=CHROMA_PATH, embedding_function=embedding_function)
```

Loads the same embedding model used before, then opens the existing Chroma database folder (rather than creating a new one, since we're not passing in documents here — just pointing at the saved data).

```
results = db.similarity_search_with_score(query_text, k=3)
```

The core retrieval step. Internally: (1) `query_text` gets embedded into a vector using `embedding_function`, (2) Chroma compares that vector against every stored chunk vector, (3) returns the `k=3` closest chunks, each paired with a distance score (lower distance = more similar meaning).

```
for doc, score in results:
    print(f"[distance={score:.4f}] {doc.page_content[:80]}...")
```

`results` is a list of (document, score) pairs — this loop unpacks each pair into two variables at once. `{score:.4f}` formats the number to 4 decimal places. `doc.page_content[:80]` uses Python slicing to grab just the first 80 characters of the chunk's text, so the debug printout stays short and readable.

```
context_text = "\n\n---\n\n".join(
    [doc.page_content for doc, _score in results]
)
```

`[doc.page_content for doc, _score in results]` is a 'list comprehension' — a compact way to build a new list by looping over results and pulling out just the text from each pair. The underscore in `_score` is a Python convention meaning 'I'm intentionally not using this value.' `"\n\n---\n\n".join(...)` then glues all three chunk texts together into one big string, with a visual '---' divider between them, so the LLM can see all three retrieved chunks as one combined context block.

```
prompt_template = ChatPromptTemplate.from_template(PROMPT_TEMPLATE)
prompt = prompt_template.format(context=context_text, question=query_text)
```

`from_template()` loads our template string into a reusable template object. `.format()` then fills in the `{context}` and `{question}` placeholders with the actual retrieved text and actual question, producing the final prompt string that will be sent to the LLM.

```
model = ChatGroq(model="openai/gpt-oss-20b", api_key=os.environ["GROQ_API_KEY"])
```

Creates a connection to Groq's API. `model="openai/gpt-oss-20b"` picks which hosted model to use. `os.environ["GROQ_API_KEY"]` reads your key (loaded earlier by `load_dotenv()`) out of the environment variables — square-bracket access here means 'this key MUST exist,' which is exactly the `KeyError` you hit earlier when the `.env` file wasn't set up yet.

```
response = model.invoke(prompt)
```

Sends the finished prompt to Groq's servers over the internet and waits for the model to generate a reply. This is the **ONLY** line in the entire project that makes a network call to an AI model — everything before it (loading, chunking,

embedding, searching) is local computation.

```
sources = [doc.metadata.get("source", "unknown") for doc, _score in results]
```

Another list comprehension, this time pulling the 'source' field (the original PDF filename) out of each retrieved chunk's metadata. `.get("source", "unknown")` safely returns "unknown" instead of crashing if that metadata field happens to be missing.

```
print(Fore.GREEN + Style.BRIGHT + response.content)
```

`response.content` holds the actual generated answer text from the model. `Fore.GREEN` and `Style.BRIGHT` are colorama codes prepended to the string, so the terminal renders this line in bright green — visually separating the real answer from the plain debug output above it.

3. Interview Quick-Reference

Q: What problem does RAG solve?

LLMs only know what they were trained on. RAG lets a model answer questions about documents it has never seen, by retrieving relevant text at question-time and feeding it to the model as context.

Q: What's the difference between retrieval and generation?

Retrieval is pure math/search — finding the most relevant chunks of text via vector similarity. No language model is involved. Generation is where an LLM reads the retrieved chunks + the question and writes a natural-language answer.

Q: What is an embedding?

A vector (list of numbers) representing the meaning of a piece of text. Texts with similar meaning get vectors that are numerically close together.

Q: Why chunk the document instead of embedding the whole PDF at once?

Embedding models and vector search work best on small, focused pieces of text. Chunking also lets you retrieve just the relevant paragraphs instead of dumping an entire book into every prompt.

Q: Why must the same embedding model be used for both indexing and querying?

Different embedding models place text in different, incompatible vector spaces. Comparing vectors from two different models is meaningless — like comparing GPS coordinates from two different map projections.

Q: What is a vector database, and why Chroma specifically?

A database optimized for storing vectors and quickly finding the closest ones to a query vector. Chroma is free, open-source, and runs locally with no server setup required — good fit for a zero-cost project.

Q: Why Groq instead of OpenAI?

Groq hosts open-weight models (like Llama or OpenAI's open gpt-oss models) on custom fast chips and offers a generous free API tier. OpenAI's models are proprietary and pay-per-use with no free tier.

Q: What does 'strict mode' mean in this project?

The prompt instructs the model to answer only from the retrieved PDF context, and to say so plainly if the answer isn't in the context, rather than filling gaps with its own general knowledge.

Q: What bug did you run into, and how did you find and fix it?

`similarity_search_with_relevance_scores()` applies a 0-to-1 normalization tuned for OpenAI's embedding distribution. With a different (HuggingFace) embedding model, real matches scored below the hardcoded 0.5 cutoff and were wrongly discarded. Fixed by switching to `similarity_search_with_score()` (raw distance, no assumption-laden normalization) and removing the arbitrary threshold.